The Sum of Our Technology

AI Changes Both Sides of Software Engineering

Vitalii Oborskyi · Editorial draft R34 · 24 October 2026

What if AI acceleration fails outside a code generator? Two shifts in software engineering demand evidence, sustainable delivery and viable socio-technical control.

Contents

When ideas outlive their authors

I. How We Build: Rebalancing the SDLC

- We accelerated a step. What did the system gain?
- Preserve What Review Does
- Verify the Explanation
- Rehearse the Recovery
- Which understanding can we replace?
- More software is not more value
- The user's capacity is part of the product

II. What We Build: Engineering Control Around Model Judgment

- Enforce the boundary before delegation
- From a passing score to a justified release
- Test shared errors between models
- Contain consequences before action
- Design for real review capacity
- Reassess authority when conditions change

III. What If Not? From Assumptions to Project Decisions

- Further reading

Sources and publication record

Sources & references

How to cite this draft

Review provenance

Revision history



The Sum of Our Technology — cover illustration. AI-generated illustration selected by Vitalii Oborskyi (18 October 2026).

An agent opens a pull request. The tests are green. The explanation is fluent. Another arrives before anyone has understood the first.

The dashboard calls this progress. The person handling the next incident has a different question.

What if not?

What if faster generation does not improve delivery—or leaves us unable to recover what we built? What if the person entrusted with the final decision cannot meaningfully make it?

Engineering starts with the problem, not the tool. My UA Online PMDay 2026 Autumn talk connects two higher-order tasks: rebalancing the SDLC without losing its essential functions, and engineering dependable control around consequential model judgment. **Both change what leaders must require, verify, staff and fund before implementation.**

Throughout this essay, **"What if not?"** is a deliberate refrain, not a forecast that every failure will occur. Each turn challenges an attractive assumption; together, they reveal the breadth and depth of the problem space before we rush to solutions. The point is practical: decide what to verify, constrain, fund or simplify.

Slides (PDF) [1] · Recording (Ukrainian) [2]

When ideas outlive their authors

Writing let ideas outlive their authors; printing extended their reach. Research on printing and the Reformation [3] traces consequences beyond books: technology changes collective capability, not only task speed.

The title echoes Lem's *Summa Technologiae* [31]. LLMs extend delegation to interpretation, design and explanation. "Delegating thinking" describes practical work, not consciousness; living with its consequences is the engineering question.

An agent can build conventional software; a human team can build a product whose runtime decisions depend on a model. The two problems are distinct.

> "The more technology takes care of us, the more carefully we must take care of it."

— Stanisław Lem; translated from Polish for this article. Publisher-supplied excerpt: [4].

I. How We Build: Rebalancing the SDLC

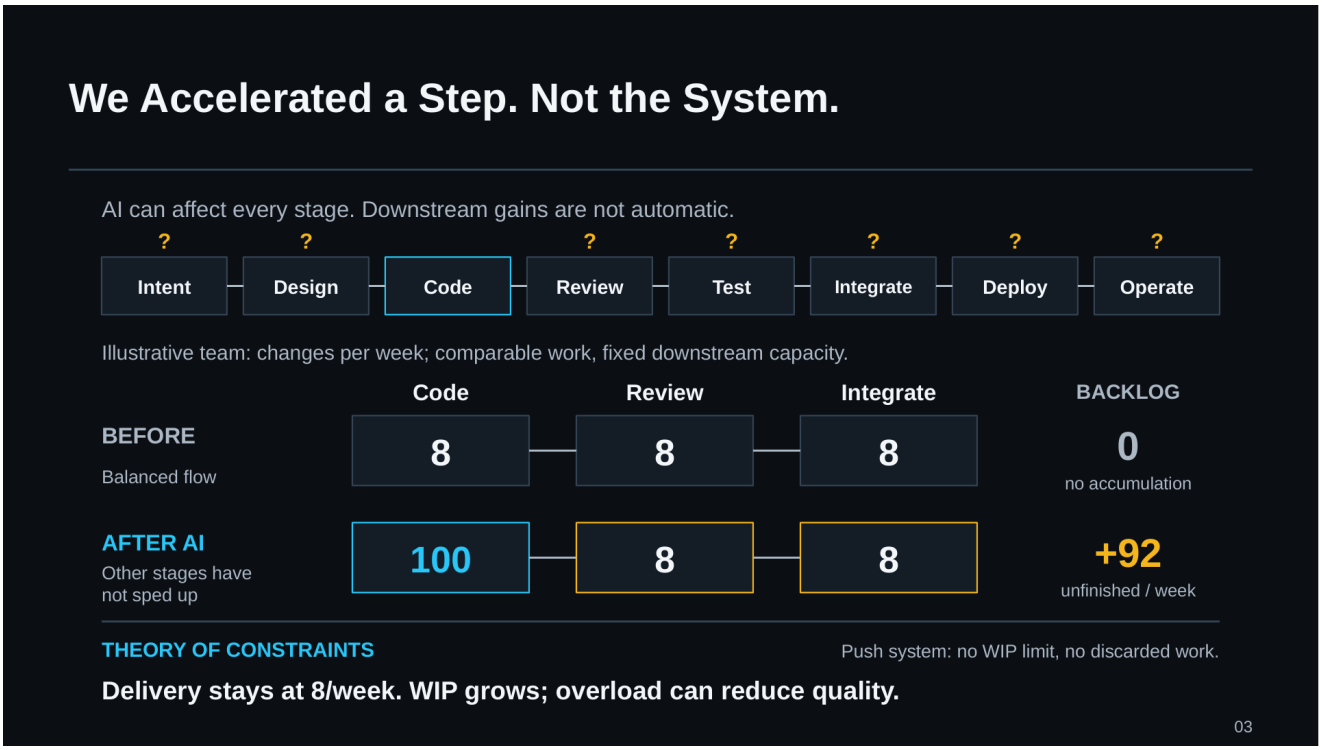
Rebalancing the SDLC is a higher-order systems-engineering problem. The task is to rebalance a chain of dependent activities without losing the functions each contributes. Code review, for example, detects defects **and** preserves shared understanding of how the system works. Removing its delay while discarding that understanding is no improvement.

The hard asymmetry is between machine output and human capacity. Agents can produce changes faster than people can assess their implications. We want to delegate cognitive work, yet teams remain accountable for the machine-made decisions embodied in what they ship. That need not mean reading every generated line: it means retaining—or replacing with demonstrably adequate mechanisms—the evidence, understanding, authority and recovery capability responsible operation requires.

We accelerated a step. What did the system gain?

Delivery requires a chain:

Intent → requirements → design → code → review → test → integration → deployment → operation.



Fixed-capacity illustration, not a reproduction of the studies' outcomes. Uncontrolled intake grows the queue; controlled intake caps it. Vitalii Oborskyi, UA Online PMDay 2026 Autumn, slide 3 (v35). [1]

Suppose a team generates, reviews and integrates eight comparable changes a week. Raise generation capacity to one hundred; keep the others fixed. Admit every change: delivery stays at eight; unfinished work grows by ninety-two a week. **We become dramatically better at producing unfinished work.** Control intake and the queue need not grow, but delivery remains capped. Spare downstream capacity would change the result.

The September revision of *Writing Code vs. Shipping Code* [7], covering more than 500,000 GitHub developers, summarises cumulative estimates as **25.5× changed lines, 3.4× commits and 1.3× releases**. These observational, outcome-specific estimates are not a conversion funnel. Changed lines include additions and deletions, not necessarily retained code or human reading time.

The roughly 30% release gain matters. Would it justify additional review, maintenance and recovery costs? These multipliers cannot answer for your team. Dependencies, duplication and deferred cleanup may bring costs later. DORA [8] associates adoption with higher throughput—and greater instability.

The NBER weak-link model describes complementary upstream and downstream work, not a universal bottleneck. If automation expands downstream capacity **while preserving verification, shared understanding and recovery**, the imbalance can disappear. Changes filtered out before review need not accumulate in its queue. [7]

Local developer productivity is a different measure. METR's early-2025 randomised trial [34] found experienced developers took 19% longer with AI but believed it made them 20% faster. Its February 2026 update [35] suggests selective participation may understate acceleration; its magnitude remains uncertain. METR's survey [18] reports median *perceived* speed gains of 3× and work-value gains of 1.4–2×, not direct observations. None establishes end-to-end delivery gains for a particular team.

Cognition introduced Devin [5] in 2024 as the "first AI software engineer". In March 2025, Dario Amodei forecast AI writing 90% of code within three to six months [6], potentially essentially all within a year. Those dates passed; the industry-wide share remains unverified. The Claude Code team reports about 90% [12]: a vendor-team report, not independent corroboration or a measure of all engineering.

The Subprime Code Crisis [9] is my synthesis of costs shifted into review, QA, architecture, maintenance and production. The danger is booking local gains before accounting for displaced work and risk.

Preserve What Review Does

Chen and Stratton [36], studying 718 firms, estimate **49% longer PR submission-to-merge time associated with AI agents**; they find no statistically significant effect of AI assistants on the examined review measures. These observational intervals include waiting and rework, not just active reviewer effort.

Anthropic's Code Review announcement [10] addresses review pressure while leaving approval to people. Microsoft's Jay Parikh [11] urges builders not to let review become a bottleneck. First-pass review moves to agents; people retain architectural oversight. He also warns that shorter shipping cycles leave less time to uncover bugs.

What if not?

Suppose defect detection improves, but incident investigation becomes harder. The approval button survives; the knowledge that gave approval meaning does not. **The queue has cleared. The ownership gap has not.**

Anthropic reports [12] that merged Claude Code PRs more than doubled while the reverted share fell over 40% between June and September 2026. Approvals bind to a specific commit; agents cannot fill required human approval slots. These internal results do not establish causality or portability, and a lower reverted share need not mean fewer reversions. The team acknowledges gaps in formal shared-understanding processes.

Maintainers need design context, teams shared knowledge, operators incident awareness and users usable workflows—not identical levels of detail.

In a matched-control study, Agarwal, He and Vasilescu [13] estimate increases in SonarQube's Cognitive Complexity of 34.85% for agent-first repositories and 42.87% for those with prior AI-IDE exposure, controlling for lines of code. GitClear [14] reports an 81% increase in its normalised duplicated-block measure from 2023. These are warning signals, not measurements of human comprehension or proof of AI causation.

Studies disagree on how reliably Cognitive Complexity predicts human comprehension [32] [33]. Interacting changes can still compound cognitive load; structural metrics remain proxies, not measures of team understanding.

Tools, abstractions and learning can raise comprehension capacity. The operating limit is reached when the team cannot explain critical behaviour or diagnose and contain failures in time.

Review transfers knowledge and builds team awareness [15]. Automating defect detection does not automatically reproduce those outcomes. Bainbridge's *Ironies of Automation* [16] describes the older trap: automation leaves people difficult exceptions while reducing the practice needed to handle them.

Verify the Explanation

Ask an LLM to explain the code. Perhaps specifications and generated documentation become durable artefacts, implementation a replaceable detail.

What if not?

Its story can inherit the implementation's mistake. Consider a team building an account-recovery assistant. In this hypothetical policy, sending a link to an already verified address is allowed; replacing that address with a newly supplied one requires separate identity proof. The agent treats both operations as equivalent. Generated code permits the replacement; tests built from the same interpretation expect success. **Code, tests and explanations can agree because they share the same mistaken interpretation.**

Can a maintainer trace the policy, permissions and fallback, then diagnose a failed recovery? Ground explanations in exact revisions, observed traces and independently checked requirements. Part II follows this mistake into production. **Understanding is a demonstrated capability, not a quantity of Markdown.**

Rehearse the Recovery

Parikh's "Code is disposable" [11] encourages rewriting bounded features whose specifications may endure. If implementation fails, the agent can supply a substitute.

What if not?

Customers cannot recover their accounts. Two proposed fixes fail. A rewrite looks tempting, but data and integrations cannot be discarded. Who stops retries, contains damage and verifies recovery?

Rewriting code does not erase obligations to customers or consequences for their data.

Rehearse recovery without the preferred agent: limit retries, preserve diagnostics and name an authorised owner. Measure containment, restoration and data integrity.

Which understanding can we replace?

A better architecture may make some forms of human understanding unnecessary.

Suppose a bounded component has an executable contract, independently grounded checks, reproducible traces and a tested fallback. Automated verification might provide stronger assurance than a hurried human reading every diff. Requiring that review anyway could add delay without useful control. Parikh's durable-specification proposal [11] and Anthropic's layered workflow [12] point towards this possibility; they do not establish it for every system.

For the recovery assistant, maintainers might no longer need to read every generated branch. They would still need to justify the identity policy, know what the checks establish, recognise failures outside their coverage and operate the fallback. A contract cannot establish that its own assumptions match the customer's needs.

Replace manual inspection where verified mechanisms cover the relevant risks. Preserve or rebuild the judgment and intervention needed for the risks they leave open. Consider the substitution when evidence shows these functions are preserved or improved.

Compare with the workflow actually operated, including its failures. Test smaller changes, intake limits and AI-assisted review against review age, rework, stability, recovery and full operating cost. Faster approval with slower recovery is not an unqualified improvement.

More software is not more value

Suppose delivery keeps up. More ideas reach customers sooner. We expect more value.

What if not?

Features multiply; the customer's problem remains. Faster implementation does not establish that the underlying assumption was right.

Cheap experiments should help reject weak ideas. Kohavi and colleagues [17] report that only about one-third of well-designed Microsoft experiments improved their intended metric. Deciding not to ship can be a successful outcome.

NBER [7] finds more new apps without commensurate growth in use. Its platform-specific proxies cannot distinguish constraints on quality and market fit from constraints on discovery and attention, or establish AI causation.

The user's capacity is part of the product

Suppose every feature is useful. We expect customers to benefit.

What if not?

A customer uses the reporting tool monthly; its workflow changes weekly. Each improvement makes sense. Each visit still requires relearning the job. Delivery has accelerated. Attention and learning time have not.

The comprehension problem has crossed the product boundary—from builders to users. Teams need enough understanding to maintain what they ship; customers need enough time and context to absorb changes that ask them to work differently.

Feature-fatigue research [19] examines how capabilities attractive before use can undermine usability afterwards. Better defaults and repaired defects, by contrast, can remove work.

Measure task completion, adoption, time to benefit and support burden. Sequence changes people must learn; invest in accessibility, reliability and fewer steps.

Competitive advantage is not how much change we deliver. It is how much better customers can work because of it.

II. What We Build:
Engineering Control Around Model Judgment

What We Build is also a higher-order engineering problem, not a routine model-integration project. Within business, risk and operating constraints, we must construct a sufficiently dependable control system around probabilistic, partly opaque model judgment whose failure modes may be incompletely characterised.

The engineered object is the entire arrangement: model, policy, tool permissions, independent checks, observation, fallback and people with the capacity and authority to intervene. Model capability alone cannot justify operational bounds or full cost. Those require evidence that errors can be detected, contained and corrected in time—and reassessment when conditions change.

The account-recovery assistant now interprets requests, retrieves guidance and proposes actions. Even without permission to change access, its guidance can influence someone who has it. Read-only does not mean consequence-free.

Traditional control, security and human-factors principles still apply. Consequential model judgment changes how they must be justified: open-ended inputs admit semantic errors, and changed models, retrieval or providers can invalidate previous evidence without an application release. In my *Uncertainty Architecture* [20], a *Thinking System* [21] has consequential runtime responsibilities partly dependent on such judgment. This is a scope definition, not a claim of consciousness or a proven universal architecture.

Six assumptions need continued testing.

Enforce the boundary before delegation

Approved guidance and prompt prohibitions are meant to prevent invented steps, restricted disclosure and bypassed authorisation.

What if not?

An attacker plants instructions in retrieved material, posing as recovery policy and urging the assistant to skip identity checks. It treats source content as authority. This hypothetical attack follows Greshake and colleagues' indirect prompt-injection mechanism [22].

A prompt prohibition is not an enforced boundary.

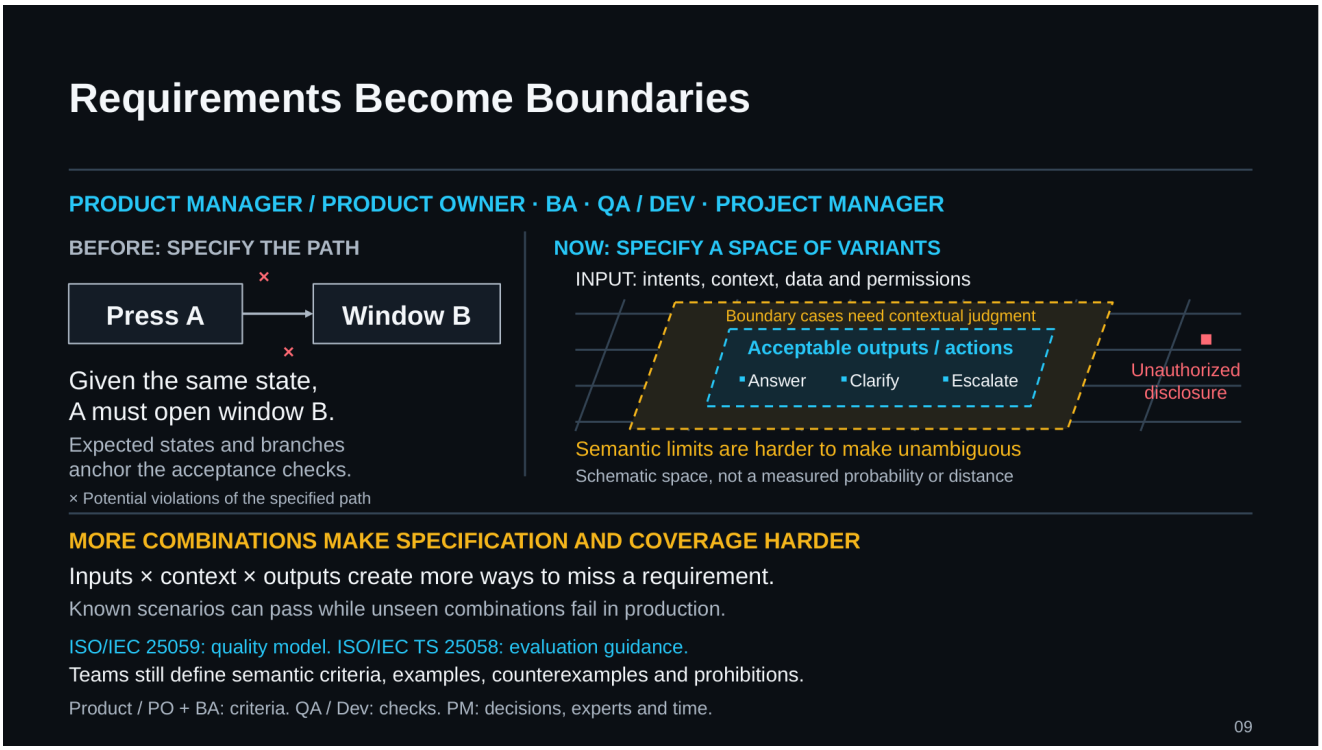
- Specify when to answer, clarify or escalate.
- Restrict retrieval and enforce permissions outside model judgment. Retrieved text cannot grant itself policy status.
- Test hostile inputs.

OWASP [23] likewise emphasises trust boundaries and least privilege, without promising immunity.

A deterministic production guard can inherit the development error, too: it permits a new recovery address without the separately required identity proof. It is independent of the runtime model but shares the mistake.

Have the policy owner settle the rule against authorised policy, then derive the forbidden case independently of the generated specification: reject a new address without separate identity proof. Test the actual access-change path. Changes to guards or expected outcomes require separate approval; the development agent cannot approve its own chain of changes. This checks that error, not policy or test-set completeness.

The diagram contrasts a specified transition with a space of admissible responses. The team must decide which answers, clarifications and escalations are acceptable—and how to check that boundary.



A specified transition and an admissible response space—not a probability map. Conventional software also has semantic requirements; model judgment makes unforeseen inputs especially important. Vitalii Oborskyi, UA Online PMDay 2026 Autumn, slide 9 (v35). [1]

From a passing score to a justified release

An illustrative evaluation uses **200 synthetic responses**:

Observed outcome	Responses	Consequence to assess
Acceptable	196	Meets the requirements under the evaluated conditions
Incorrect recovery instructions	3	May cause rework, lockout or unsafe action; severity remains to be assessed
Restricted information disclosed	1	Violates the example’s explicit prohibition

Ninety-eight percent acceptable. The tempting conclusion: ready to take responsibility.

What if not?

The percentage is correct; the release decision is wrong. Under the rule prohibiting disclosure, the candidate fails. Other responses cannot cancel that consequence. Nor does pooling all failures as 2% make their severity equivalent. [24]

Observing zero failures in 3,000 independent, representative trials still leaves a one-sided 95% upper bound near 0.1%, assuming reliable classification and stable conditions. [24] A curated Golden Set need not represent production. Ordinary traffic cannot establish resistance to an attacker selecting and adapting inputs; test those threats separately.

Evaluation supplies evidence. Severity, coverage, uncertainty and enforceable limits determine what it justifies delegating.

Test shared errors between models

A second model catches errors the generator misses. The combined path should be more dependable.

What if not?

Both accept the same misleading document and unsupported premise. A second verdict does not establish independent verification.

Knight and Leveson [25] found correlated failures in independently implemented programs—a historical warning, not a measurement of modern LLMs. Test shared-error cases alongside valid outputs and verified failures. Measure missed errors and unnecessary blocks; enforce permissions separately. Timeout or uncertainty must trigger a defined wait, denial or fallback, not authorisation.

Contain consequences before action

Telemetry alerts the owner; rollback reverses the change. We expect correction fast enough to contain consequences.

What if not?

The alert arrives after disclosure or after an employee follows unsafe instructions. Reverting the model erases neither. **A working recovery mechanism is not the same as a recoverable consequence.**

- Enforce critical limits before action.
- Observe outputs, denials, fallbacks, reviewer queues and control health.
- Give an authorised owner the ability to narrow or stop operation and verify the result.

These operational responsibilities are reflected in NIST's AI RMF [28]. If intervention cannot beat the consequence, restrict authority, redesign or reject the path.

Design for real review capacity

"Human in the loop" promises qualified judgment and intervention before harm.

What if not?

The queue is unattended at night. At peak load, reviewers cannot keep up. A tired reviewer approves a polished summary without seeing the evidence. The path exists. The capacity does not.

Parasuraman and Manzey [26] connect automation complacency to attention and competing tasks; experience is no guarantee. Bainbridge's trap returns: difficult exceptions reach someone with diminished practice. [16]

If safe operation depends on human judgment, human capability is an architectural dependency.

- Supply competence and access to evidence.
- Staff the review path and give reviewers the power to refuse.
- Measure queue wait plus decision time against the intervention window, including peak demand.

An escalation policy cannot staff a queue. Human response capacity is an operating cost and release condition.

Uncertainty Architecture's control-loop anatomy [27] supplies my framing: a socio-technical stack, not software plus oversight.

The application denies account-access changes until a qualified person approves the specific action. Without a reviewer, it also suppresses model-generated recovery advice, displaying only a fixed, preapproved route to the verified recovery channel. The model cannot add steps or expose account-specific information. Case-specific recovery waits for the staffed process; waiting never becomes approval.

This bounded release shows where delegation must stop, **not that AI adds value**. Test data isolation, fixed output and receiving capacity. A static page could offer the same route; useful AI delegation needs separate evidence.

Reassess authority when conditions change

Evidence, controls, human capacity and fallback support bounded operation. We can rely on that arrangement while its conditions hold.

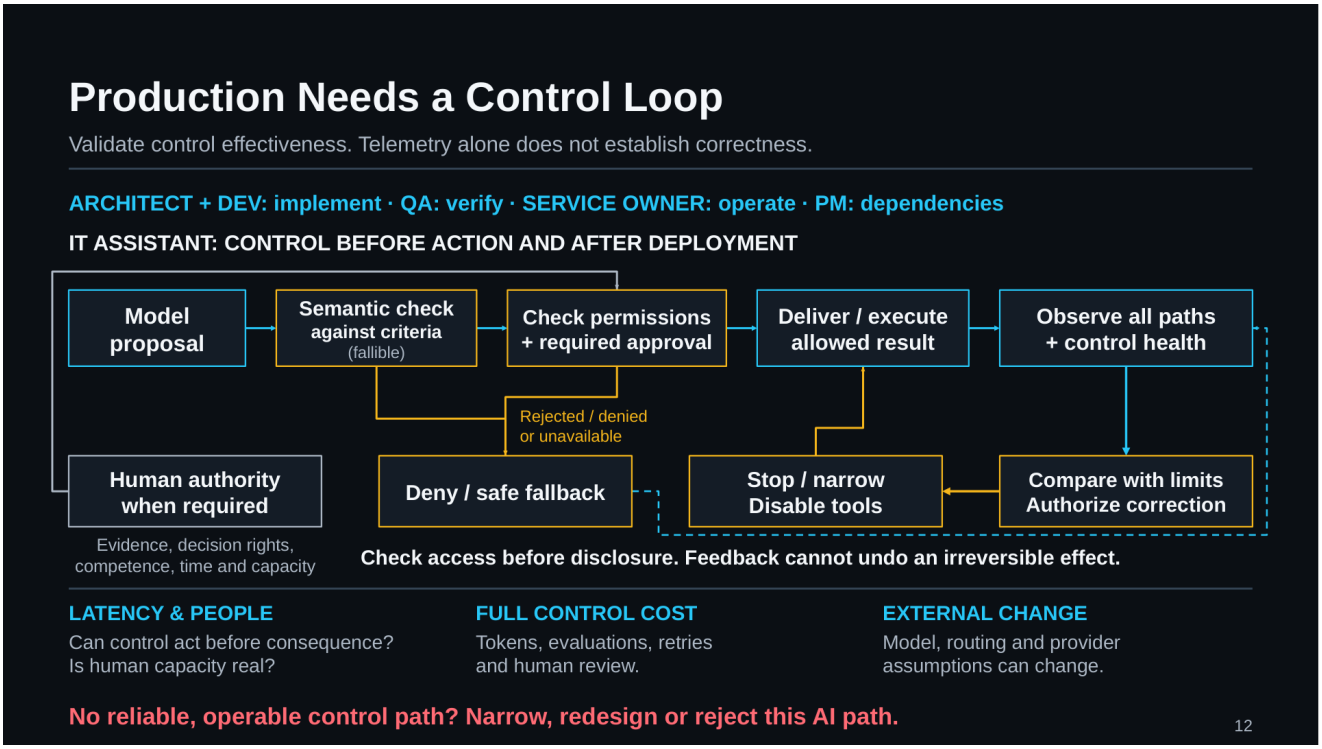
What if not?

The model or provider routing changes, retrieval acquires conflicting material, or demand overwhelms reviewers. No application commit is required for yesterday's evidence to stop describing today's system.

Record supported model, data, permissions, workload and staffing conditions, with reassessment triggers. Turn incidents into regression cases; retest controls and capacity; revisit authority when its justification changes.

Release authorises operation under particular conditions. It cannot make those conditions permanent.

Read the loop as conditions for action: a proposal must pass the relevant checks and permissions, while observation covers delivery, denial and fallback. Correction needs an authorised owner and enough capacity to act; the fallback and the controls themselves also require testing.



Permissions, evidence, fallible semantic checks and verified intervention form one operating path. Human capacity and response time must be supplied and tested. Vitalii Oborskyi, UA Online PMDay 2026 Autumn, slide 12 (v35). [1]

III. What If Not? From Assumptions to Project Decisions

The first investment decision already depends on the problem statement. Approving coding agents without assessing review capacity, shared understanding, maintenance and recovery leaves the delivery problem incomplete. Approving an agentic product without specifying authority, validation, intervention, fallback and full operating cost leaves the control problem incomplete. Both omissions shape the project before implementation starts.

Product narratives understandably highlight speed, autonomy and model capability. Those component gains matter, but do not establish system performance. **Before implementation**, identify actual constraints and critical functions, set boundaries and evidence thresholds, and budget human capacity and recovery. The diagnosis changes requirements, acceptance criteria, risk and staffing.

Compare the AI path with both the current process and the simplest credible safe alternative under the same safety requirements. Use comparable cases, data and staffing, with the same task, identity requirements and definition of successful recovery.

Does the model improve completed recoveries, completion time or unnecessary transfers enough to justify its full cost? Include maintenance, support and oversight. Clicking a link is not recovery; gains from the shared downstream process do not establish the model’s contribution.

A routing gain is not proof of useful recovery or permission for broader action.

For the recovery team, these **illustrative pilot limits** turn that framing into decisions. Set criteria before collecting results; these are not measured findings or universal thresholds.

Decision	Evidence and limit	Action if the limit fails
Increase AI-assisted implementation	Predeclare meaningful improvement in median change-to-release time, tolerable deterioration in p90 review age and escaped defects per comparable change, and sufficient observations. Four weeks is an initial baseline, not a sample-size guarantee. Maintainer passes a recovery rehearsal.	Cap intake if limits fail. Continue the bounded pilot without expansion if evidence is insufficient. Generated lines do not justify expansion.
Permit account-specific advice or action	Verify access boundaries and approval before advice/action. Independently derived policy cases must reject the new-address error through the actual access-change path. Predefine populations, failure classes and justified risk limits. Report trial/failure counts; require one-sided 95% upper bounds to meet limits where sampling assumptions hold. Test adversarial coverage separately.	Prohibited disclosure, bypass or unapproved change rejects the candidate. Insufficient evidence keeps routing only; 0/3,000 is not a release rule.
Rely on a reviewer	Approval precedes case-specific advice/action. Exercise a ten-minute timeout at peak load: a service deadline, never permission for unsafe output.	Fixed, preapproved routing only; no account-specific information.
Recover control	Rehearse stopping, diagnostics and data-integrity checks. Observed prohibited disclosure or access change stops the broader path immediately.	Contain the incident; rollback cannot undo disclosure.
Expand the pilot	After four weeks, compare useful recovery completions, time, support load and full cost with the real process and simplest safe alternative under the same safety requirements. Require the predeclared useful gain and passing control tests. Reassess after model, retrieval, permission or staffing changes.	No useful gain: choose the simpler path. Insufficient evidence: retain bounds and extend observation. An uneventful month cannot establish rare-event safety; routing gains do not authorise broader advice/action.

Faster implementation does not authorise broader runtime judgment.

Canary releases [29] make exposure, evaluation and expansion explicit. Agile's principles [30] connect delivery with technical excellence and sustainable pace. Both support learning within limits; neither makes every consequence reversible.

Vendors owe adopters evidence, clear limits and visibility into consequential changes. Their capacity to absorb risk does not establish what your customers can accept.

> "One cannot simultaneously make discoveries and avoid taking responsibility for their consequences."

— Stanisław Lem, quoted and translated by Bogna Konior; source and edition details: [31].

"What if not?" becomes an acceptance criterion, a funded dependency and tested recovery. The sum of our technology includes the systems, institutions and human capabilities needed to live with what we delegate.

Build for the future you believe in. Keep the ability to act if that belief is wrong.

Further reading

The Subprime Code Crisis [9] and *Uncertainty Architecture* [20] develop the two strands: my synthesis and developing architectural doctrine, not independent experimental confirmation of this essay.

Sources & references

Numbered citations refer to the sources below. Source notes retain the evidence qualifications of the preceding draft. The discussion of substituting verified controls for implementation-level inspection, the hypothetical policy error and the proposed product comparison are architectural arguments, not new empirical findings.

1. Vitalii Oborskyi. AI Changes the Delivery System and the Product Itself. UA Online PMDay 2026 Autumn presentation. See the Slides (PDF) link at the beginning of this article.

   Figures reproduce full slides 3, 9 and 12 from the v35 presentation copy; the complete deck remains linked above. The event name follows the organiser's announcement. Original slide credits and notices remain applicable. Scenarios and proposed countermeasures are not empirical results.

2. Vitalii Oborskyi. UA Online PMDay 2026 Autumn talk recording (Ukrainian).

3. Jared Rubin. "Printing and Protestants: An Empirical Test of the Role of Printing in the Reformation." Review of Economics and Statistics 96(2), 270–286 (2014).

4. Stanisław Lem. Fantastyka i futurologia. Quotation selected in Wiktor Jaźniewicz's compilation, W strachu przed antymaterią. 1381 cytatów (Wydawnictwo Literackie, 2024).

   The Polish passage is reproduced under "TECHNOLOGIE JAKO TAKIE" in a publisher-supplied excerpt published by Instytut Spraw Obywatelskich on 22 July 2025, and attributed there to Fantastyka i futurologia—not Summa Technologiae. The English wording in the prologue is a translation prepared for this article from that Polish excerpt, not a quotation from a published English edition. No original-volume page number is supplied by the excerpt.

5. Cognition. "Introducing Devin, the first AI software engineer" (12 March 2024).

   The announcement also describes collaboration and human review; it is not a literal promise to replace all programmers immediately.

6. Council on Foreign Relations. CEO Speaker Series: Dario Amodei (10 March 2025).

   The 10 March 2025 forecast implied June–September 2025 for 90% and around March 2026 for essentially all code. Those dates have passed. This essay records the forecast, not a verified industry-wide share or a verdict on fulfilment. The team report in [12] comes from the same vendor and is not independent corroboration. The same discussion retains human specification, design, integration and security responsibilities.

7. Mert Demirer, Leon Musolff and Liyuan Yang. "Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools." NBER Working Paper 35275 (2026; September revision). · Author-hosted full text

Table 6, printed p. 33: weeks 21–30, pre-period-normalised, winsorised event-study estimates. Cumulative multipliers: lines 1 + (234.3 + 957.5 + 1254.9)/100 = 25.467; commits 3.441; releases 1.288 (no separate async estimate). Outcome samples differ. Figure 1 rounds these to 25.5, 3.4 and 1.3. Section 6.3 calibrates the weak-link model to the autocomplete pattern, giving substitution elasticity about 0.23; this is a model-dependent estimate, not a measured universal review constraint. Agents can intervene at later layers, so the gap may narrow. Section 6.2 describes experimentation filtered downstream; discarded work need not be valueless. Marketplace §§7.2.3–4, pp. 45–47 use first-three-month cohort proxies, not all installed software or welfare: iOS ratings are roughly stable, Android downloads rise modestly and Chrome downloads fall. The authors cannot distinguish supply-side quality/polish/market-fit constraints from consumer discovery/attention constraints. Neither is established as the sole mechanism. Observational working paper, not peer-reviewed. Section 4.1 combines public GitHub activity with confidential Microsoft GitHub Copilot usage/subscription data to date autocomplete and GitHub Sync adoption and validate asynchronous-agent signals. Demirer and Musolff disclose former Microsoft postdoctoral positions and current paid research consulting for Microsoft on the NBER page.

8. Jessica Baolin and Nathen Harvey / DORA. “Balancing AI Tensions: Moving from AI Adoption to Effective SDLC Use” (10 March 2026; analyses the 2025 report).

   Survey associations, not randomised causal estimates.

9. Vitalii Oborskyi. The Subprime Code Crisis — independent research synthesis.

   Author-led synthesis, not independent experimental validation.

10. Anthropic. “Bringing Code Review to Claude Code” (9 March 2026).

    First-party product announcement and internal usage claims, not independent effectiveness evidence. It explicitly leaves PR approval to humans. Its March product description is distinct from the October team's risk-tiered workflow in [12].

11. Jay Parikh / Microsoft. “Introducing Command Line, and the new rules for builders” (2 June 2026).

    First-party engineering guidance, explicitly provisional. The opening supplies the old-playbook instruction; sections 8–10 cover first-pass agent review with human architectural oversight, shorter bug-detection buffers and rewriting bounded features whose specifications may endure. The opening also discusses post-deployment learning, which is related to but not identical with consequential runtime judgment.

12. Addy Osmani, Morgan Westlee Lunt and Qing Wang / Anthropic. “How the Claude Code team does code review” (15 October 2026).

    First-party team account: about 90% Claude-authored code and the June–September comparison are self-reported, not an industry share or an external causal/portability study. Reverted share and absolute volume differ. Described practices include layered checks, risk-tiered approvals, staged release, approval bound to a commit and no substitution of an agent for a required human reviewer. Fixing one's own small/medium change is usually a team norm. The authors explicitly acknowledge gaps in formal shared-understanding processes and automatic tightening of approvals.

13. Shyam Agarwal, Hao He and Bogdan Vasilescu. “AI IDEs or Autonomous Agents? Measuring the Impact of Coding Agents on Software Development.” arXiv:2601.13597v2 (27 January 2026).

    Table 2; §3.2 includes lines of code as a covariate. §4 notes limitations including isolated pre-treatment differences. Not a per-line or human-comprehension metric. Reported Cognitive Complexity estimates are 34.85% for agent-first repositories and 42.87% for those with prior AI-IDE exposure.

14. GitClear. The AI Code Quality & Maintainability Gap (2026).

    Normalised duplicated-block measure versus the 2023 baseline, not a raw block count; observational vendor report. This is distinct from GitClear's short-window rewrite measure of code churn.

15. Alberto Bacchelli and Christian Bird. “Expectations, Outcomes, and Challenges of Modern Code Review.” ICSE (2013), 712– 721.

16. Lisanne Bainbridge. “Ironies of Automation.” Automatica 19(6), 775–779 (1983). · Full text (IFAC)

    Foundational human-factors analysis, not an LLM deployment trial.

17. Ronny Kohavi, Thomas Crook, Roger Longbotham, Brian Frasca, Randy Henne, Juan Lavista Ferres and Tamir Melamed. “Online Experimentation at Microsoft” (2009). · Full text

    §5–5.1. The one-third result describes the authors’ Microsoft experience, not a universal feature-success rate.

18. Joel Becker / METR. “Measuring the Self-Reported Impact of Early-2026 AI on Technical Worker Productivity” (11 May 2026).

    Survey of 349 respondents; convenience-sample self-reports. Median perceived speed gain: 3×; work-value gains: 1.4–2×. The range reflects different question wording, not a confidence interval.

19. Debora Viana Thompson, Rebecca W. Hamilton and Roland T. Rust. “Feature Fatigue: When Product Capabilities Become Too Much of a Good Thing.” Journal of Marketing Research 42(4), 431–442 (2005).

    Does not establish a human learning rate. The monthly-user/weekly-update example in this essay is hypothetical. Daily experts and occasional users may face different learning costs.

20. Vitalii Oborskyi. Uncertainty Architecture — open doctrine and pattern language.

    Author-led doctrine and pattern language; not independent empirical validation.

21. Uncertainty Architecture. “Uncertainty in the Controlled Object.”

    Selected draft-normative definition. Consequential does not by itself mean high risk or production readiness.

22. Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz and Mario Fritz. “Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.” arXiv:2302.12173v2 (5 May 2023).

Demonstrates attacks through retrieved, attacker-influenced content in tested LLM-integrated applications. The account- recovery example in this essay is hypothetical, not a reported incident; the paper does not establish an attack success rate for current models.

23. OWASP Gen AI Security Project. “LLM01:2025 Prompt Injection.”

    Security guidance on direct/indirect injection, least privilege, trust boundaries and adversarial testing. Mitigation guidance is not a guarantee of immunity or evidence that this example's controls have been deployed.

24. NIST/SEMATECH. e-Handbook of Statistical Methods, §7.2.4.1, Confidence intervals for a proportion.

    Author calculations for the synthetic example: 1/200 gives a 95% Wilson interval of 0.0883%–2.7774%; 4/200 gives about 0.8%–5.0%. For zero failures, the exact one-sided 95% binomial upper bound is 1 − 0.05^(1/n); at n = 3,000 it is 0.0998%, close to the “rule of three”. Independence, representative sampling, reliable classification and stable conditions are assumptions—not release guarantees.

25. John C. Knight and Nancy G. Leveson. “An Experimental Evaluation of the Assumption of Independence in Multiversion Programming.” IEEE Transactions on Software Engineering SE-12(1), 96–109 (1986).

    Historical programming experiment, not a quantitative estimate of modern LLM error correlation.

26. Raja Parasuraman and Dietrich H. Manzey. “Complacency and Bias in Human Use of Automation: An Attentional Integration.” Human Factors 52(3), 381–410 (2010). · Abstract

    Research review of attention and automation-related decision errors in novice and expert users.

27. Uncertainty Architecture. “Control-Loop Capability Anatomy.”

    Selected draft-normative architectural framing, not field evidence that a particular human-control arrangement works.

28. NIST. Artificial Intelligence Risk Management Framework (AI RMF 1.0), Core (2023).

    Operational risk-management framework, not certification of the example system.

29. Alec Warner and Štěpán Davidovič, with Alex Hidalgo, Betsy Beyer, Kyle Smith and Matt Duftler. “Canarying Releases.” The Site Reliability Workbook, chapter 16 (2018).

    Defines canarying as partial, time-limited deployment plus evaluation to guide rollout. Its discussion of exposure, detection and isolation supports bounded learning, not a guarantee that all consequences can be reversed.

30. Manifesto for Agile Software Development. “Principles behind the Agile Manifesto” (2001).

    Connects agility with technical excellence, good design and sustainable development. Not a licence to omit consequences analysis or release without appropriate controls.

31. Stanisław Lem. Summa Technologiae (1964). · Quotation source: Bogna Konior, “Existential Technologies,” Antikythera.

The responsibility quotation in part III uses Konior's own English translation. Her note 18 cites the Polish edition (Kraków: Wydawnictwo Literackie, 2020), p. 471. It was checked in her essay, not independently against that book page.

32. Marvin Muñoz Barón, Marvin Wyrich and Stefan Wagner. "An Empirical Validation of Cognitive Complexity as a Measure of Source Code Understandability." ESEM (2020). · DOI

    Meta-analysis of ten studies, 427 code snippets and approximately 24,000 evaluations. Finds associations with comprehension time and subjective ratings; correctness and physiological results are mixed. The metric weights nested control flow. It does not establish a universal nonlinear relationship between a percentage change in the score and human cognitive load.

33. Luigi Lavazza, Sandro Morasca and Marco Gatto. "An empirical study on software understandability and its dependence on code characteristics." Empirical Software Engineering 28, 155 (2023).

    Study of three master's students performing maintenance tasks on 32 methods from two applications. Structural metrics correlate with task time, but prediction accuracy is limited; Cognitive Complexity does not improve it in this setting. Small sample. The discussion of disproportionate effort from interacting conditions and dependencies is an engineering explanation of a risk, not a measured scaling law from this study.

34. Joel Becker, Nate Rush, Beth Barnes and David Rein / METR. "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity" (10 July 2025).

    Randomised AI-allowed/disallowed tasks for 16 experienced developers on familiar repositories; 246 tasks, chiefly using Cursor with Claude 3.5/3.7 Sonnet. The 19% longer completion time is an early-2025 result, not an estimate for all developers or current tools. The 20% perceived speedup was elicited after participation.

35. Joel Becker, Nate Rush, Tom Cunningham, David Rein and Khalid Mahamud / METR. "We are Changing our Developer Productivity Experiment Design" (24 February 2026).

    Later point estimates suggested shorter task times, but confidence intervals included no improvement. METR says refusal to work without AI and selective task submission likely bias the speedup estimate downwards, and believes acceleration likely increased by early 2026. Changed compensation and concurrent-agent time accounting add uncertainty. These data remain weak evidence of the increase's magnitude; the direction of likely selection bias is not a proven current effect size.

36. Fiona Chen and James Stratton. "Artificial Intelligence in the Firm: Bottlenecks in Software Production" (August 2026).

    Working paper using proprietary Jellyfish data from 718 consenting client firms, January 2021–March 2026. Staggered difference-in-differences design, not randomised assignment. The agent estimate in §4.1 is 3.45 additional days (standard error 1.60) from PR submission to merge against a 7.03-day baseline, approximately 49%. This elapsed interval includes waiting and other work; it does not measure active reviewer effort. Jellyfish supplied aggregated/anonymised metadata, not code content, and reviewed confidentiality rather than scientific content. Sample and adoption-selection limits constrain generalisation.

How to cite this draft

Oborskyi, V. (2026). *The Sum of Our Technology: AI Changes Both Sides of Software Engineering*. Editorial draft R34, 24 October 2026.

Unpublished author-review edition. Cite the revision; no DOI or final publication URL has been assigned.

```
@misc{oborskyi2026sumR34,
  author = {Oborskyi, Vitalii},
  year = {2026},
  title = {The Sum of Our Technology: AI Changes Both Sides of Software Engineering},
  note = {Editorial draft R34, 24 October 2026; unpublished}
}
```

Text © 2026 Vitalii Oborskyi. Presentation and third-party credits remain applicable. Cover 02: AI-generated editorial illustration selected by the author.

Review provenance

Maximiliano Armesto suggested clarifying how policy mistakes propagate from development into production controls and comparing AI with simpler alternatives under the same safety requirements. Both recommendations informed **R24**.

Pasquale Langella contributed a critical reading and an AI-assisted alternative draft. Selected suggestions on reader orientation, navigation and practical responses were incorporated in **R25**.

Olivier Leturcq contributed feedback on the balance between innovation and sustainable operating costs, and suggested examining how architecture affects the context needed for human review and maintenance. These comments are recorded for editorial consideration.

Vitalii Oborskyi retains editorial responsibility.

Revision history

24 October 2026:

- **R34:** Moved METR productivity findings to the SDLC evidence discussion and restored the cross-boundary comprehension argument for users.
- **R33:** Sharpened the two system-level problems for engineering leaders, qualified evidence and bounded delegation, and connected problem framing to project decisions without expanding scope.
- **R32:** Sharpened the two higher-order engineering problems: SDLC rebalancing under machine–human speed asymmetry, and bounded control around probabilistic model judgment. Linked both to early project framing and planning.
- **R31:** Restored the “What if not?” refrain as a deliberate rhetorical device to reveal the problem space and turn challenged assumptions into engineering decisions.

- **R30:** Combined problem-led headings, consolidated repeated arguments and clarified project decisions; native print hierarchy and grouping improved.
- **R29:** Foregrounded both problems and their planning implications.
- **R28:** Engineering problem framing clarified across delivery, runtime control and project planning.

23 October 2026:

- **R27:** Olivier's feedback recorded; substantive editorial proposals remain under review.
- **R26:** Clickable citations; concise provenance and history.
- **R25:** Selected Pasquale editorial suggestions incorporated.
- **R24 addendum:** Pasquale's review and alternative draft recorded.
- **R24:** Maximiliano's policy-error and simpler-alternative feedback incorporated.
- **R23 addendum:** Maximiliano's feedback added to R23 (21 October).